Agentic AI

agent = model + tools + a loop

Contents

page	updated
2. What is an AI agent	2026-10-20
3. Which framework to pick	2026-10-20
4. TypeScript SDKs	2026-10-20
5. Python frameworks	2026-10-20
6. Full app + agent	2026-10-20
7. Prompts	2026-10-20
8. Sources	2026-10-20

last updated 2026-10-20

What is an AI agent

updated 2026-10-20

Core idea

- **Agent = AI model + tools + a loop**
- It keeps going: think → use a tool → read the result … until the job is done
- Every framework is just a different way to write this same loop

The one loop

```
User task → Your app (CLI / web / bot) → Agent (model + system prompt)
                                                  |
                                 needs a tool? ├─ yes → run tool (API, file, DB, search)
                                                  |          └─ result goes back to the model (loop)
                                                  └─ no  → final answer → user
```

Remember

- The model only **decides**; your code **runs** the tools
- Loop stops when the model gives a final answer (or a step limit hits)
- Read: Anthropic – *Building effective agents*

Which framework to pick

updated 2026-10-20

I want ...	Use	Lang
Claude Code's power (files, bash, edit) in my app	**Claude Agent SDK**	TS + Py
Small agents that hand work to each other	**OpenAI Agents SDK**	TS + Py
Any model in a web app, switch in 1 line	**Vercel AI SDK**	TS
Full control: steps, branches, memory, approval	**LangGraph**	Py (+JS)
A quick team of role-based agents in YAML	**CrewAI**	Py

Quick rule

- Web app (Next.js) → **Vercel AI SDK**
- Coding / file / ops agent → **Claude Agent SDK**
- Long, stateful, human-in-the-loop → **LangGraph**
- Fast multi-agent demo → **CrewAI**

TypeScript SDKs

updated 2026-10-20

Setup (all): Node 20+ → `npm init` → install SDK → API key in `.env` → `src/index.ts`

Claude Agent SDK

- `npm i @anthropic-ai/claude-agent-sdk`
- `for await (msg of query({ prompt, options }))`
- Options: `model, allowedTools, systemPrompt, permissionMode`
- `CLAUDE.md` = rules · `.claude/agents/` = sub-agents · `.mcp.json` = outside tools
- Built-in tools: Read, Edit, Bash, Grep, WebSearch

OpenAI Agents SDK

- `npm i @openai/agents zod`
- `new Agent({ name, instructions, tools })` → `await run(agent, input)`
- Triage agent with `handoffs: [a, b]` routes to specialists
- Guardrails + tracing built in

Vercel AI SDK

- `npm i ai @ai-sdk/google zod`
- `new ToolLoopAgent({ model, instructions, tools, stopWhen })`
- `app/api/chat/route.ts` → `agent.stream()` · UI → `useChat()`
- Change model in one line (google / anthropic / openai / ollama)

Python frameworks

updated 2026-10-20

Setup (all): Python 3.10+ → `uv init` → `uv add <pkg>` → key in `.env` → `uv run main.py`

Claude Agent SDK

- `uv add claude-agent-sdk`
- `async for msg in query(prompt, ClaudeAgentOptions(allowed_tools, system_prompt))`
- Custom tools: `@tool` + `create_sdk_mcp_server`

OpenAI Agents SDK

- `uv add openai-agents`
- `Agent(name, instructions, handoffs=[...])` → `Runner.run(triage, msg)`
- `@function_tool` — its **docstring is the tool description**

LangGraph

- `uv add langgraph langchain`
- `State(TypedDict)` → nodes `call_model` / `call_tools` → `StateGraph(State)`
- `.add_conditional_edges` (tool call? → tools : END) → `.compile(checkpointer)`
- Most control; can pause for human approval (interrupt)

CrewAI

- `uv tool install crewai` → `crewai create crew my_crew`
- `agents.yaml` (role, goal, backstory) + `tasks.yaml` (description, expected_output)
- `crewai run` → `kickoff(inputs={...})`

Full app + agent

updated 2026-10-20

The agent is **one part** of your app: its own folder, called through **one API route**.

Setup	Fits	Like
(a) TS monorepo: `apps/web` + `packages/agents`	Claude / OpenAI / Vercel SDK	—
(b) Next.js frontend + FastAPI agent backend (SSE)	LangGraph, CrewAI	Mitra AI
(c) Mobile app + agent API on Vercel	any TS SDK	Stack

Flow

```
User → app UI → /api/agent (checks login) → runAgent() → tools (only this user's data)
     ← answer streams back (SSE) ←
```

5 rules

1. Model API keys **only on the server** — never in browser or APK
2. Agent code in its own folder → easy to test + swap model
3. App passes the logged-in **user id** to tools — the model never chooses whose data
4. Stream answers (SSE); long jobs → worker / queue
5. Shared types (zod / Pydantic) in one place

Prompts

updated 2026-10-20

A. Building with an AI tool — R G C S R D

1. **Role** – who the AI acts as
2. **Goal** – what you want at the end
3. **Context** – stack, folders, docs
4. **Steps** – "plan first, show me, then code"
5. **Rules** – what NOT to do
6. **Done** – how we know it's finished (tests pass)

B. Inside the agent's code

```
<role>You are the support agent for Stack.</role>
<goal>Solve it in as few steps as possible.</goal>
<tools>Call get_order BEFORE any order answer.</tools>
<rules>Never invent data. Ask one question if info is missing.</rules>
<output>2-3 short lines + next step</output>
```

- **Tool description** = WHEN to use it + what each input means (model only sees this text)
- **Structured output** = JSON via zod / Pydantic
- **Routing** = small triage prompt picks the agent

Most used

plan-first · few-shot examples · prompt chaining · "find the cause first" · "list only real bugs"

Sources

updated 2026-10-20

- Anthropic — Building effective agents
- Anthropic — Effective context engineering
- Anthropic — Writing tools for agents
- Claude Agent SDK docs
- OpenAI Agents SDK quickstart
- Vercel AI SDK — Agents
- LangGraph overview
- CrewAI docs